

bulk_register_aips ❖❖❖ Manuale

Direzione ICT, Università❖❖ degli Studi di Milano

Luglio 2026

Contents

Manuale — bulk_register_aips	1
1. Scopo	1
2. Come funziona, passo per passo	1
3. Il path “a quadranti” (pairtree)	2
4. Fallback di identificazione via METS	3
5. Cross-check di coerenza UUID	3
6. Report JSON	4
7. Configurazione (.env)	4
8. Dopo la registrazione: indicizzazione nel Dashboard	4
9. Troubleshooting	4
10. Note di sicurezza	5

Manuale — bulk_register_aips

1. Scopo

Questo strumento risolve un problema specifico: hai una cartella con AIP (Archival Information Package) che esistono fisicamente sul filesystem, ma che la Storage Service di Archivemantica non conosce — cioè non hanno un record nella sua tabella dei package (Package). Senza quel record, l’AIP:

- non compare tra i risultati di ricerca “Archival Storage” del Dashboard,
- non può essere scaricato, verificato (fixity) o cancellato tramite l’interfaccia o l’API della Storage Service,
- non è considerato “gestito” dal sistema di preservazione.

Lo script automatizza il confronto tra ciò che è **fisicamente presente** su disco e ciò che è **logicamente noto** alla Storage Service, e registra la differenza (gli AIP “orfani”) con una chiamata API per ciascuno.

Questo script si occupa **solo** della registrazione nella Storage Service. Per farli comparire anche nella ricerca del Dashboard, dopo l’esecuzione bisogna rigenerare l’indice Elasticsearch degli AIP (vedi sezione 8).

2. Come funziona, passo per passo

2.1 Inventario fisico

`find_local_aips()` scansiona ricorsivamente `--scan-dir` con `os.walk`, quindi trova i file a qualunque livello di annidamento, indipendentemente da come sono organizzate le sotto-

cartelle.

Per ogni file trovato, cerca di determinarne l'UUID in due modi:

1. **Dal nome del file**, se rispetta il pattern <nome>-<uuid>.<estensione> (es. mio-dataset-3fa85f64-5717-4562-b3fc-2c963f66afa6.7z). Questo è il caso più comune per AIP prodotti normalmente da Archivematica.
2. **Dal METS interno**, se il nome del file non contiene un UUID riconoscibile ma l'estensione è un formato di archivio supportato (.7z, .tar.gz, .tar.bz2, .tar, .zip). Utile per AIP prodotti da strumenti come **a3m** o **DART3**, o per file rinominati manualmente. Vedi sezione 4.

2.2 Inventario logico

`get_registered_aip_uuids()` interroga (con paginazione) `GET /api/v2/file/?package_type=AIP` sulla Storage Service e raccoglie tutti gli UUID già registrati come package di tipo AIP.

2.3 Differenza

`find_orphan_aips()` confronta i due inventari e distingue:

- **orfani “sicuri”**: presenti su disco, non registrati, nessuna incoerenza rilevata tra nome file e METS interno;
- **conflitti**: presenti su disco, non registrati, MA con UUID nel nome file diverso da quello nel METS interno (vedi sezione 5). Questi **non** vengono registrati automaticamente.

2.4 Registrazione

Per ogni AIP orfano “sicuro”, `register_aip()` esegue una `POST /api/v2/file/` alla Storage Service con un payload che include: `uuid`, `origin_location/origin_path` (dove si trova ora fisicamente), `current_location/current_path` (dove deve risultare, vedi sezione 3), `package_type`: "AIP", `size`.

Se `origin_path` e `current_path` coincidono, la Storage Service registra il record senza spostare fisicamente nulla. Se sono diversi, tenta uno spostamento reale dei file dalla posizione attuale a quella di destinazione.

3. Il path “a quadranti” (pairtree)

Archivematica organizza gli AIP dentro una location di tipo Local Filesystem secondo uno schema deterministico basato sull'UUID, per evitare di avere decine di migliaia di file in un'unica cartella piatta:

<4hex>/<4hex>/<4hex>/<4hex>/<uuid-completo>/<nome-file>

dove i quattro blocchi da 4 caratteri sono i primi 16 caratteri esadecimali dell'UUID (senza trattini). Esempio, per l'UUID 3fa85f64-5717-4562-b3fc-2c963f66afa6:

3fa8/5f64/5717/4562/3fa85f64-5717-4562-b3fc-2c963f66afa6/mio-dataset-3fa85f64-5717-4562-b3fc-2c963f66afa6.7z

Questo schema dipende **solo dall'UUID**, non dall'istanza Archivematica che lo genera: due Archivematica diverse, per lo stesso UUID, calcolano esattamente lo stesso path.

`resolve_current_path()` sfrutta questo fatto per decidere cosa fare:

- se il percorso relativo trovato su disco **coincide già** con quello atteso (tipico di un AIP che arriva da un'altra istanza Archivematica, già organizzata a quadranti) → lo riusa così com'è, **nessuno spostamento fisico**;

- se **non coincide** (tipico di un AIP “flat” da a3m/DART3, mai stato organizzato secondo questo schema) → calcola il path a quadranti atteso e lo passa come `current_path`, così la Storage Service **sposta fisicamente** il file nella posizione standard.

Importante: questo confronto è relativo a `--scan-dir`. Se punti `--scan-dir` esattamente alla radice della struttura a quadranti (es. il path della location AIP Storage di un’altra istanza), il confronto funziona correttamente. Se invece scandisci una cartella che *contiene* quella radice con un livello di annidamento in più, l’auto-detect non riconoscerà la struttura esistente e tratterà l’AIP come se dovesse essere spostato (il risultato finale sarebbe comunque corretto, ma la label nel log dirà “spostamento fisico” invece di “nessuno spostamento” — se questo ti succede, ripeti lo scan puntando alla vera radice).

Verifica consigliata prima di un run massivo: prendi un AIP che sai essere stato registrato normalmente da Archivematica (non da questo script) e controlla il suo `current_path` reale con:

```
curl -k -H "Authorization: ApiKey archivematica:LA_TUA_API_KEY" \
https://192.168.139.39:8000/api/v2/file/<uuid-noto>/
```

e confronta con quanto calcolato da `compute_quadrant_path()`, per assicurarti che coincidano sulla tua installazione specifica.

4. Fallback di identificazione via METS

Quando il nome del file non contiene un UUID riconoscibile (es. AIP da a3m/DART3 con convenzioni di naming diverse, o file rinominati), lo script apre l’archivio **senza estrarlo interamente** e cerca l’UUID autoritativo nel METS:

1. Cerca un’entry con nome `METS.<uuid>.xml` — è il caso più comune, e basta leggere l’elenco dei contenuti dell’archivio (veloce, non serve leggere il contenuto del file).
2. Se non trovata, cerca un’entry che sembra un METS (`mets*.xml`), ne legge il contenuto e cerca l’UUID nell’attributo `OBJID` dell’elemento `<mets:mets>`.

Formati supportati nativamente in Python: `.zip`, `.tar`, `.tar.gz`, `.tar.bz2`. Per `.7z` è necessario il binario `7z` (pacchetto `p7zip-full`) installato e nel `PATH` del sistema dove gira lo script: se non è disponibile, viene stampato un avviso e quell’AIP viene ignorato dalla scansione (a meno che il nome del file non contenga già un UUID valido).

Questo comportamento si può disattivare con `--no-mets-fallback` (più veloce su collezioni molto grandi, ma ignora gli AIP con nomi non standard e disabilita anche il cross-check descritto sotto).

5. Cross-check di coerenza UUID

Anche quando il nome del file **contiene già** un UUID valido, lo script apre comunque l’archivio (se `--no-mets-fallback` non è passato) e confronta quell’UUID con quello scritto nel METS interno. Il motivo: il nome del file potrebbe essere stato alterato manualmente, o l’AIP potrebbe essere una copia rinominata di un altro, e in quel caso l’UUID “giusto” da usare per la registrazione è quasi certamente quello nel METS, non quello nel nome.

Quando i due non coincidono, l’AIP viene marcato come **conflitto** e **non viene registrato automaticamente**: compare in una sezione separata dell’output e del report JSON (`conflicts`), con il proprio percorso e l’UUID trovato nel nome file, per una verifica manuale caso per caso (potrebbe trattarsi di un file rinominato per errore, di un reingest, o di una copia di test).

6. Report JSON

Al termine dell'esecuzione viene scritto un file di report (default `bulk_register_report.json`) con questa struttura:

```
{
  "scan_dir": "...",
  "dry_run": true,
  "results": {
    "<uuid>": {"success": true, "status_code": 202, "will_move": false, "uuid_source": "filename"},
  },
  "conflicts": {
    "<uuid-dal-nome-file>": {"rel_path": "...", "filename_uuid": "..."}
  }
}
```

7. Configurazione (.env)

Vedi `.env.example`. Variabili principali:

- `SS_URL`, `SS_USER`, `SS_API_KEY`: connessione alla Storage Service
- `SSL_VERIFY`: false per i certificati self-signed (come su UNIMI)
- `TARGET_LOCATION_UUID`: UUID della location AIP Storage di destinazione

8. Dopo la registrazione: indicizzazione nel Dashboard

La registrazione nella Storage Service **non** rende automaticamente gli AIP cercabili nella scheda “Archival Storage” del Dashboard, perché quella ricerca usa un indice Elasticsearch separato. Per rigenerarlo, sul server del Dashboard:

```
manage.py rebuild_aip_index_from_storage_service --delete-all
```

Opzioni utili: `--pipeline` per una pipeline diversa da quella corrente, `-u/--uuid` per un solo AIP, `--delete` per rimuovere prima le voci con quell'UUID (utile se mancano solo alcuni AIP). In versioni più recenti di Archivematica (1.18+) il comando potrebbe essere lanciato tramite `python -m archivematica.dashboard.manage` invece di `manage.py py` — verifica sulla tua installazione.

9. Troubleshooting

Sintomo	Possibile causa
[WARN] comando '7z' non trovato	installare <code>p7zip-full</code> sul server dove gira lo script
Un AIP scompare dalla scansione	il nome file non matcha il pattern e il fallback METS non ha trovato un UUID (controlla se l'archivio è corrotto o il METS ha un nome non standard)
Un AIP finisce sempre in conflicts	il METS interno riporta davvero un UUID diverso dal nome file: va verificato manualmente quale sia quello corretto prima di registrare
<code>will_move=True</code> inatteso su un AIP già a quadranti	<code>--scan-dir</code> probabilmente non punta esattamente alla radice della struttura a quadranti (vedi sezione 3)

Sintomo	Possibile causa
Errore HTTP 500 dalla Storage Service	verificare che TARGET_LOCATION_UUID sia corretto e che l'utente API abbia i permessi sulla location

10. Note di sicurezza

- Testare sempre --dry-run prima di un'esecuzione reale.
- Testare su un singolo AIP in ambiente non di produzione prima di un run massivo, specialmente se will_move=True (spostamento fisico reale).
- Il file .env contiene una API key: non va mai committato (verificare che sia escluso dal .gitignore del repo).